SQUEEZE: Two-Bit Key-Value Cache Quantization with Per-Channel Keys

Priya Raman, Oskar Weiss, Dana Feld (Corvid Systems Research)

Abstract

During long-context inference the key-value cache, not the model weights, dominates GPU memory. We present SQUEEZE, a tuning-free method that stores the cache in two bits per number. Keys are quantized per channel and values per token, following the structure of their outliers, and a small window of recent tokens is kept in full precision. SQUEEZE reduces KV cache memory by 5.3 times and raises decoding throughput by 3.1 times with an average accuracy loss of 0.4 points on LongSuite.

1 Introduction

For a 7B model serving a batch of 16 requests at 32K tokens, the cache needs 256 GB in half precision, far more than the weights. Quantizing the cache is attractive because it needs no retraining, but naive two-bit quantization destroys accuracy. We analyse why and design a quantizer around what we find.

2 Analysis of the Cache

We inspected the cached keys and values of every layer of three model families. Keys and values behave differently. In the keys, a few key channels carry outliers whose magnitude is ten to fifty times the median, and they are the same channels for every token. Values show no fixed outlier channels, but their error matters per token because the attention output is a weighted sum of value rows.

This motivates quantizing keys per channel, so that each outlier channel gets its own scale, and quantizing values per token, so that the error of each value row is bounded. Quantizing keys per token, as earlier work did, spreads one outlier's range over the whole row and leaves the other channels with almost no resolution.

3 Method

Each quantization group shares a scale and a zero point. We use a group size of 32 for both keys and values and asymmetric rounding to two bits:

$$Q(x) = \mathrm{round}((x - z) / s), \quad s = (\max - \min) / 3, \quad z = \min \qquad (1)$$

Per-channel quantization of keys needs a group of tokens to be complete before it can be quantized, so the newest tokens wait in a residual buffer: the most recent 128 tokens stay in full precision and are quantized together once the buffer is full. Attention is computed over the quantized part and the residual part and the two results are combined.

4 Results

Table 1: Memory, throughput and LongSuite accuracy, 7B model, 32K context.

Method	Bits	Memory	Throughput	LongSuite
Full precision	16	1.0x	1.0x	47.6
Per-token 2-bit	2	5.3x	3.1x	31.2
SQUEEZE	2	5.3x	3.1x	47.2

SQUEEZE keeps accuracy within 0.4 points of full precision on average while using a fifth of the memory, and the larger batches it allows raise throughput by 3.1 times. The residual buffer is essential: without it accuracy drops by 2.7 points on the retrieval tasks.

5 Discussion

Quantization reduces the size of every cached entry but keeps every entry. It is therefore complementary to methods that decide which tokens to keep, and the two can be applied together.

References

[1] G. Alvarez and M. Stone. Outlier channels in transformer activations. Transactions on Efficient Computing, 2022.

[2] Y. Kato. Asymmetric low-bit rounding for inference. arXiv preprint, 2023.